AIL 841: Assignment 2
Semester 2, 2025-26

Due 4th April. 2026 12:00 PM

Instructions:

- You should submit all your code (including any pre-processing scripts written by you) and any graphs that you might plot.
- Include **a write-up (pdf) file**, which includes a brief description for each question explaining what you did. Include any observations and/or plots required by the question in this single write-up file.
- Please use Python for implementation using only standard python libraries (e.g. numpy). Do not use any third party libraries/implementations for algorithms.
- Your code should have appropriate documentation for readability.
- You will be graded based on what you have submitted as well as your ability to explain your code.
- This assignment is supposed to be done individually. You should carry out all the implementation by yourself.
- We plan to run Moss on the submissions. We will also include submissions from previous years since some of the questions may be repeated. Any cheating will result in a zero on the assignment, a penalty of -10 points and possibly much stricter penalties (including a **fail grade** and/or a **DISCO**).
- Starter code is available at this link.
- A total of 5 buffer days are allowed across all assignments. After the exhaustion of the 5 buffer days 10% penalty will be levied for each day late.

1 Natural Policy Gradient (NPG) in Tabular MDPs [30]

In standard Policy Gradient methods (like REINFORCE), the gradient updates can suffer from slow convergence due to the geometry of the parameter space. The Natural Policy Gradient (NPG), overcomes this by pre-multiplying the gradient by the inverse of the Fisher Information Matrix (FIM).

1.1 Theoretical Derivation of Tabular NPG

Consider a tabular Markov Decision Process (MDP) where we parameterize our policy using a tabular softmax representation. For every state s and action a, we maintain an unconstrained parameter $\theta_{s,a} \in \mathbb{R}$. The policy is defined as:

$$\pi_\theta(a|s) = \frac{\exp(\theta_{s,a})}{\sum_{a'} \exp(\theta_{s,a'})} \tag{1}$$

Task: Provide a step-by-step mathematical proof demonstrating that the Natural Policy Gradient update direction for a specific state s, $\mathbf{w}_s = F_s^{-1}\mathbf{g}_s$, is proportional to the Advantage function $A^\pi(s,a)$.

- First, derive the analytical form of the score function $\nabla_{\theta_s} \log \pi_\theta(a|s)$ and construct the Fisher Information Matrix $F(\theta)$.
- Then solve the linear system $F_s \mathbf{w}_s = \mathbf{g}_s$.

1.2 Implementing Tabular NPG

Using the theoretical derivation from the previous subsection, you will now implement exact Tabular NPG and REINFORCE.

We will use the `TreasureHunt` gridworld environment (similar to Assignment 1, Question 2, but you must use the new starter code provided. Instead of estimating values with neural networks or rollouts, use the environment's exact transition matrix T and reward matrix R to compute the exact Value function V^π and Advantage function A^π at each iteration.

Task:

- Implement Tabular NPG using the simplified update rule derived in Subsection 1.1, with a fixed learning rate η.
- Implement standard REINFORCE for comparison.
- Repeat your training runs across 3 different random seeds.

1.3 The Limitations of Exact NPG

While exact NPG is highly elegant in a tabular setting, scaling it introduces severe computational and mathematical bottlenecks. Answer the following theoretical questions:

1. **The Lagrangian Step Size:** In the Trust Region formulation of NPG, we bound the KL-divergence by a strict threshold δ. This results in a dynamic learning rate $\eta = \sqrt{\frac{2\delta}{\mathbf{g}^T F^{-1} \mathbf{g}}}$. What mathematical difficulty will you face if you try to compute this exact dynamic learning rate in our tabular codebase, compared to using a fixed learning rate?
2. **Continuous Control & Deep Learning:** Suppose you are solving a continuous control environment (e.g., `Hopper-v4`) using a Neural Network policy with 10,000 parameters. What specific difficulties will you face if you attempt to exact NPG at every step?

Deliverables for Section 1

For 1.1 & 1.3 (Theory):

- Clear, step-by-step mathematical derivation of Tabular NPG.
- Written answers explaining the computational bottlenecks of the dynamic learning rate and exact Deep NPG.

For 1.2 (Coding & Experiments):

- **Training Plot:** A plot showing the L2 Norm of $V^\pi(s)$ across iterations for both NPG and REINFORCE (averaged over 3 seeds, with shaded standard deviation).
- **Evaluation Table:** After training, evaluate the final greedy policy for 100 episodes. Report a table showing the **Mean ± Std of Cumulative Reward** and the **Average Number of Treasures Collected** over the 100 episodes for both algorithms.
- Finally create the gif, one for NPG and one for REINFORCE.

2 Trust Region Policy Optimization (TRPO) from Scratch [30]

To overcome the limitations of NPG discussed in Section 1.3, Schulman et al. (2015) introduced TRPO. TRPO approximates the natural gradient direction using the Conjugate Gradient method via Fisher-Vector products, and enforces the KL-divergence constraint using a backtracking line search.

Task: Implement TRPO from scratch using PyTorch. You will evaluate your implementation on two continuous control environments from Gymnasium: `Hopper-v4` and `HalfCheetah-v4`.

- Your policy should be a Gaussian policy.

- You will need to implement the Conjugate Gradient algorithm and the Fisher-Vector product trick to compute the search direction.
- Implement a backtracking line search to ensure the surrogate objective improves and the KL constraint $D_{KL}(\pi_{old}||\pi_{new}) \leq \delta$ is strictly satisfied.
- Train your TRPO agent on both environments. Repeat the experiments for **3 random seeds**.

Deliverables for Section 2

- Training curves for `Hopper-v4` and `HalfCheetah-v4` showing Episodic Return vs. Timesteps. The curves must show the mean and standard deviation across the 3 seeds.
- Evaluation for 100 episodes after training, and report mean and std.
- GIF for each environment.

3 TRPO with Stable Baselines 3 [10]

Finally, you will compare your from-scratch implementation against an standard library. Because TRPO is highly complex, it is maintained in the `sb3-contrib` repository rather than the core Stable Baselines 3 library.

Task:

- Install the contrib package (`pip install sb3-contrib`).
- Train the TRPO algorithm on `Hopper-v4` and `HalfCheetah-v4`.
- Use a same network architecture and KL-threshold (δ) as your from-scratch implementation to ensure a fair comparison.
- Repeat the experiments for 3 random seeds.

Deliverables for Section 3

- Combined training curves comparing the Episodic Return of your **from-scratch TRPO** vs. **SB3 TRPO** for both Hopper and HalfCheetah (mean and standard deviation across 3 seeds).
- A brief analysis of the performance difference. Did SB3 perform better? If so, what standard optimizations or tricks might SB3 be using under the hood that your implementation lacks?
- GIF for each environment.

4 Conservative Policy Iteration [30]

Train an agent using Conservative Policy Iteration on the TreasureHunt-v1 environment (Fig. 1). This environment is a grid world with 4 available actions corresponding to movement in four directions. The objective is to navigate the ship from any starting position to the fort, which has a fixed location in the top-right corner, while collecting the treasure along the way and avoiding pirates. Additionally, some cells contain land, which acts as a barrier to movement. The state is represented as a four-channel binary image, $s \in \{0,1\}^{4\times10\times10}$. Each channel corresponds to a specific entity: the positions of the treasure, ship, land, and pirates are encoded with ones at their respective locations. Use MLP regressor for value function approximation. Use Monte Carlo estimates. You are free to choose the number of roll-outs, maximum horizon length for each roll-out. Collect a number of training samples, design features incorporating state information and partially fit the value function model regressing on Monte Carlo estimates of discounted cumulative reward. For roll-outs use a random restart distribution as suggested in the reference paper. Use advantage estimates to find the greedy policy. The expected runtime on CPU is of the order of 25-30 minutes for each α.

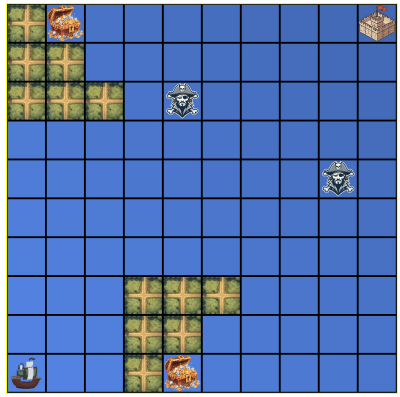

Figure 1: TreasureHunt-v1 Environment (10x10)

Deliverables for Section 4

1. Implement the standard Conservative Policy Iteration and vary the strength $\alpha = [0.01, 0.02, 0.05, 0.1]$ of mixture coefficient (see reference paper for more details). For each α after each policy improvement step, evaluate the improved policy using environment steps and record the cumulative un-discounted reward. Plot and submit the series of un-discounted cumulative rewards over training iterations.

2. For each α, after the model has been trained, evaluate the final policy using environment step functions until the agent reaches the fort or 200 steps. Plot the obtained policy using "visualize_policy" function provided in the environment code. Also plot the gif of the trained policy. For this part, you can evaluate the agent starting from state '0'.

3. Mention the hyper-parameters chosen for Monte Carlo estimates and for training the value function model in the report. Plot the heatmap for value estimates for all states after the training is complete. Include the plots in the report.

Note : Please note that there are subtle changes in the environment file with respect to previous assignment. Please use the latest version for implementation. Reference : "Approximately Optimal Approximate Reinforcement Learning" by Sham Kakade and John Langford, Proceedings of the 19th International Conference on Machine Learning (ICML 2002).